Prota — Backend Documentation

Complete backend architecture and file-by-file catalog

Phase 2A: Secure Authentication and Profile Management · Prepared for Saichu · October 20, 2026

A technical manual of the Prota backend: process flow diagrams, security and session architecture, the database schema, API contracts, and an itemized reference for every file.

Item	Detail
Release milestone	Phase 2A: Secure Authentication & Profile Management
Primary stack	Java 25 / 26, Spring Boot 4.1.1, Spring Security 7.0.9, PostgreSQL 18.4
Security guarantees	HTTP-only SameSite=Lax session cookie, double-submit CSRF, BCrypt $2a$
Test suite status	21 automated integration tests passing (0 failures, 0 skipped, 0 errors)

In this document

Section	Covers
1. Overview & Technology Stack	Stack and versions
2. Visual Process & Architecture Flow Diagrams	Architecture, sessions, AI logging, analytics, reminders
3. Backend File-by-File Catalog	Every file, class, repository, DTO and test
4. API Contract Reference	Endpoints with request and response examples
5. Error Handling	RFC 9457 error envelope
6. Developer Setup	Running and packaging the tests
7. Open Items and 8. Final Word	Points to confirm, and the next step

1. Overview & Technology Stack

Prota is engineered for security, data integrity and strict domain boundaries. It runs with full schema validation against PostgreSQL 18, with Flyway providing version control for the schema.

Layer / component	Technology / specification
Programming language	Java 25 / 26 (LTS compiler toolchain)
Framework & engine	Spring Boot 4.1.1 / Spring Framework 7.0.9
Security system	Spring Security 7.0.9 (HTTP-only session cookies, CSRF, BCrypt)
Persistence / ORM	Spring Data JPA / Hibernate 7.x (ddl-auto: validate)
Relational database	PostgreSQL 18.4
Database versioning	Flyway 11 (via spring-boot-starter-flyway)
JSON processing	Jackson 2.18 with JavaTimeModule & custom serializers
Build tooling	Apache Maven (via project Maven Wrapper .\mvnw.cmd)
Health & telemetry	Spring Boot Actuator (/actuator/health, /actuator/info)

2. Visual Process & Architecture Flow Diagrams

These diagrams show how the Prota backend processes data, enforces security and isolates user boundaries.

2.1 High-level architecture flow

Client requests pass through the Spring Security boundary, then the REST controllers, service transactions and JPA persistence before reaching PostgreSQL. Gemini and Web Push are reached only from the backend.

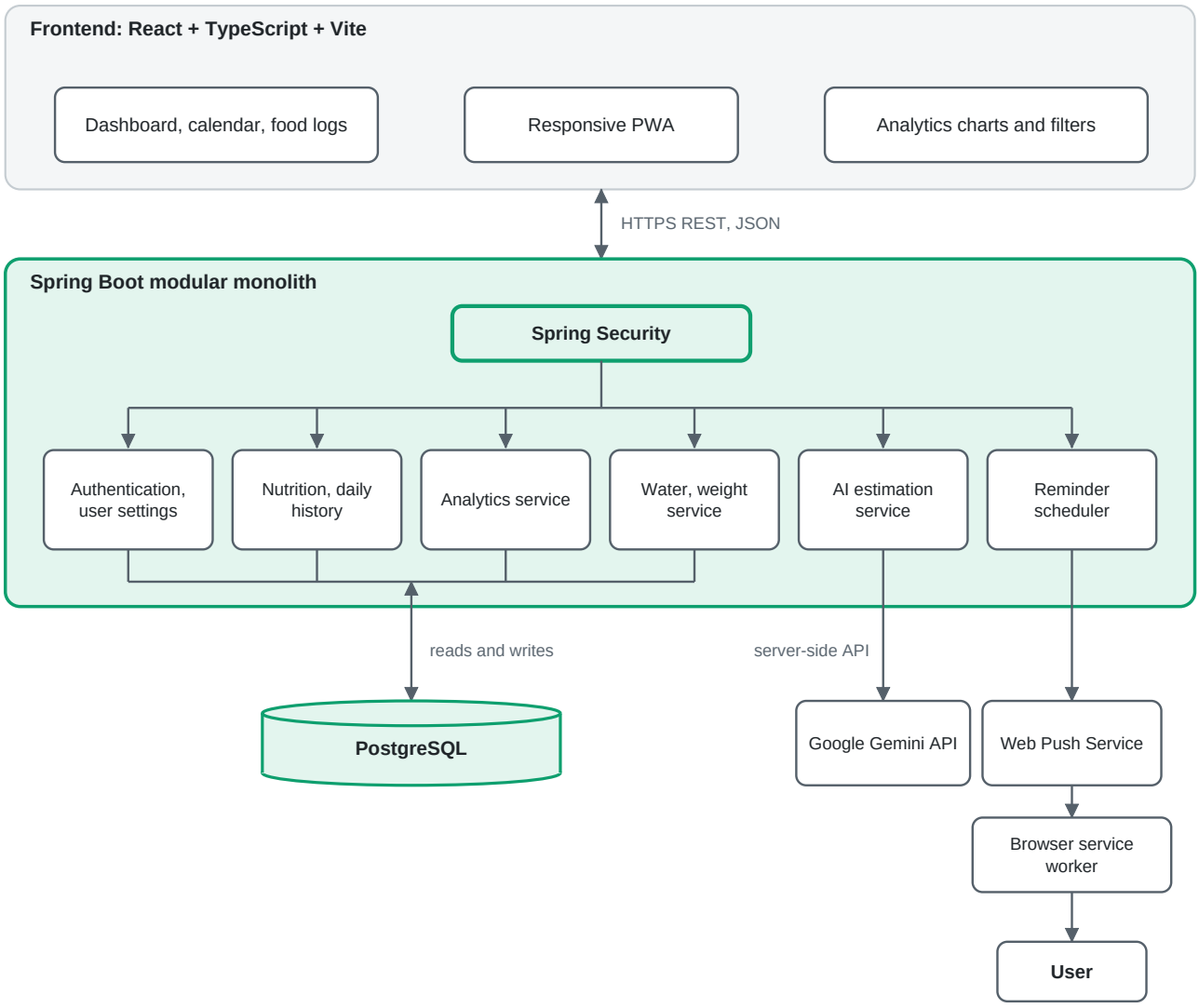

Figure 1: Prota multi-tier system architecture and gateway routing

Request path at a glance

Step	What happens
1. Browser to API	The React PWA calls the Spring Boot API over HTTPS using REST and JSON.
2. Security gate	Spring Security checks the session cookie and CSRF token before any controller runs.
3. Services	Controllers hand off to transactional services, which persist through Spring Data JPA.
4. Storage	PostgreSQL is the single source of truth for every module.
5. AI	Gemini is called only from the AI estimation service, server-side.
6. Reminders	The reminder scheduler sends through Web Push to the browser service worker and then the user.

2.2 Authentication & session lifecycle flow

Prota establishes authenticated sessions with server-managed cookies, session fixation defense and double-submit CSRF tokens.

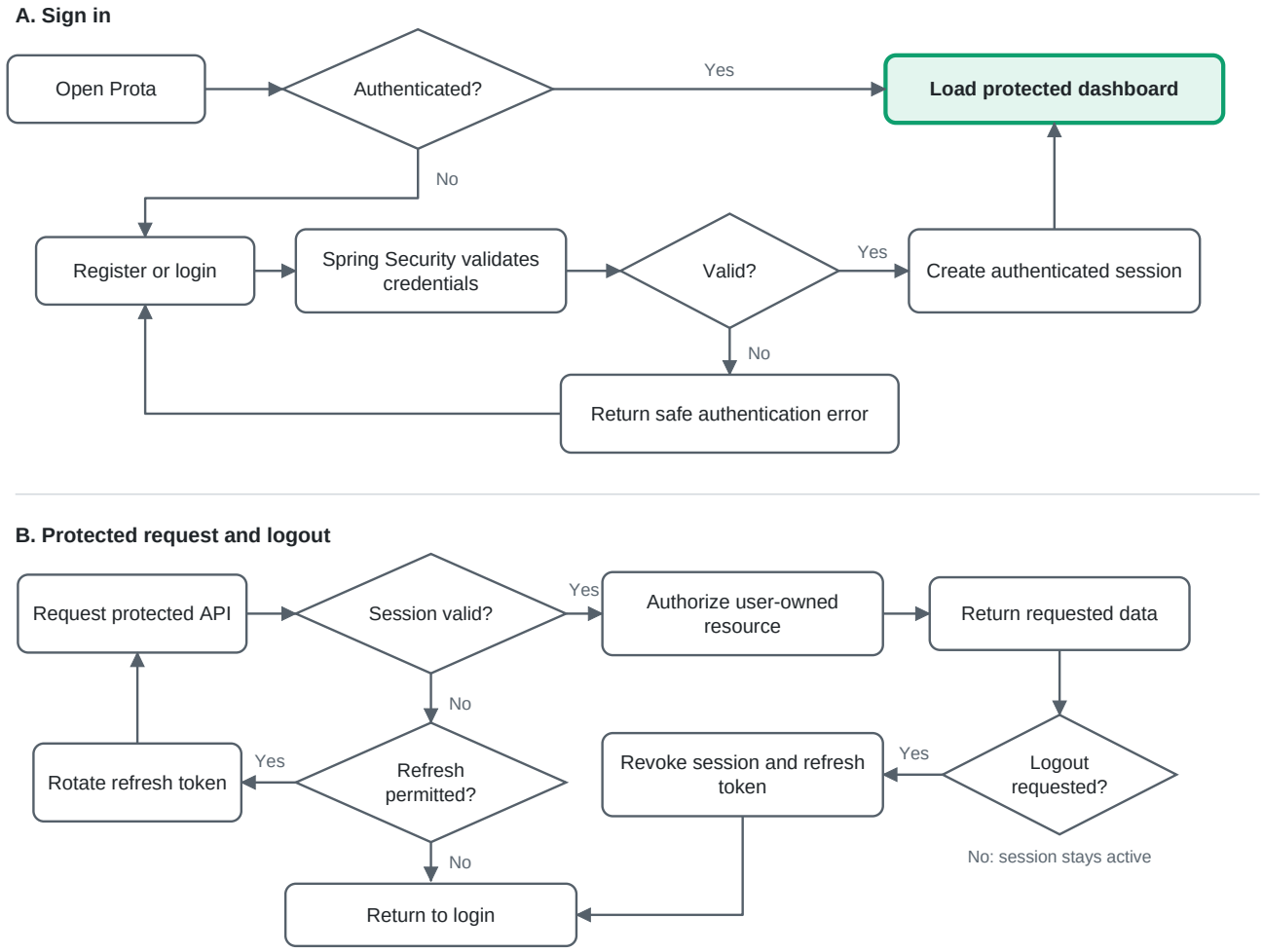

Figure 2: Registration, session rotation, cookie exchange and invalidation. Refresh-token rotation is reserved for a future release (see RefreshToken.java).

Session guarantees

Control	How it works
Session cookie	PROTASESSIONID, HTTP-only, SameSite=Lax.
Session fixation	The session ID changes on login (changeSessionId).
CSRF	Double-submit token through CookieCsrfTokenRepository. The SPA reads it from GET /api/v1/auth/csrf and sends it in the X-XSRF-TOKEN header.
Passwords	Hashed with BCrypt ($2a$). Responses never include the hash.
Logout	Invalidates the session and clears SecurityContextHolder (HTTP 204).
Refresh tokens	Reserved for a future release; sessions use the server-side cookie today.

2.3 AI food logging & multimodal estimation flow

The AI pipeline accepts text or an image, stages estimates in AiFoodEstimate, asks the user to confirm, and only then persists entries into the DailyLog. Manual entry stays available as a fallback.

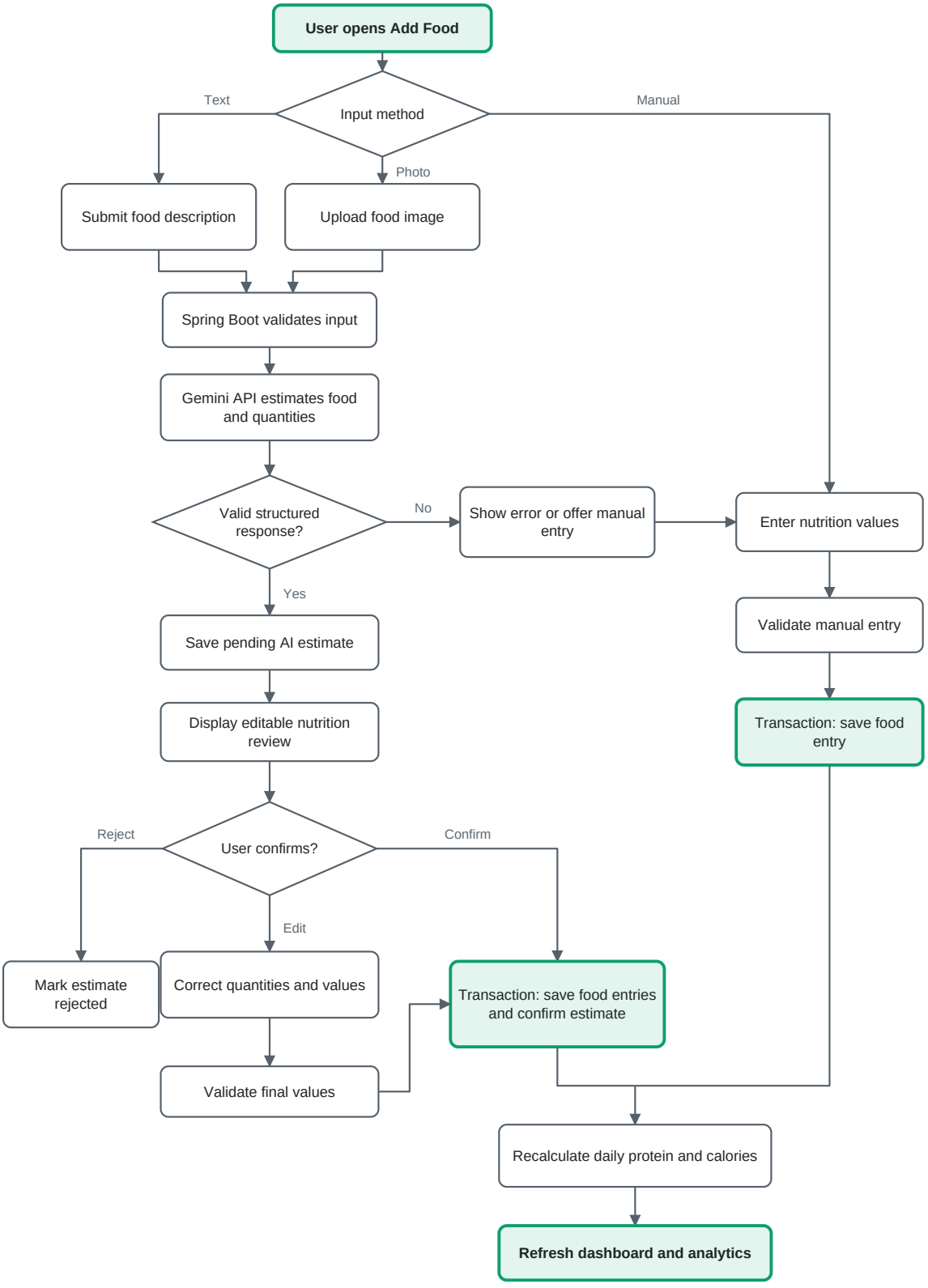

Figure 3: Multimodal food parsing, AI estimation and confirmation flow

2.4 Analytics, streaks & historical rollup flow

Daily totals are computed dynamically. Updating user settings never alters historical DailyLog goal snapshots, which keeps streaks and analytics accurate.

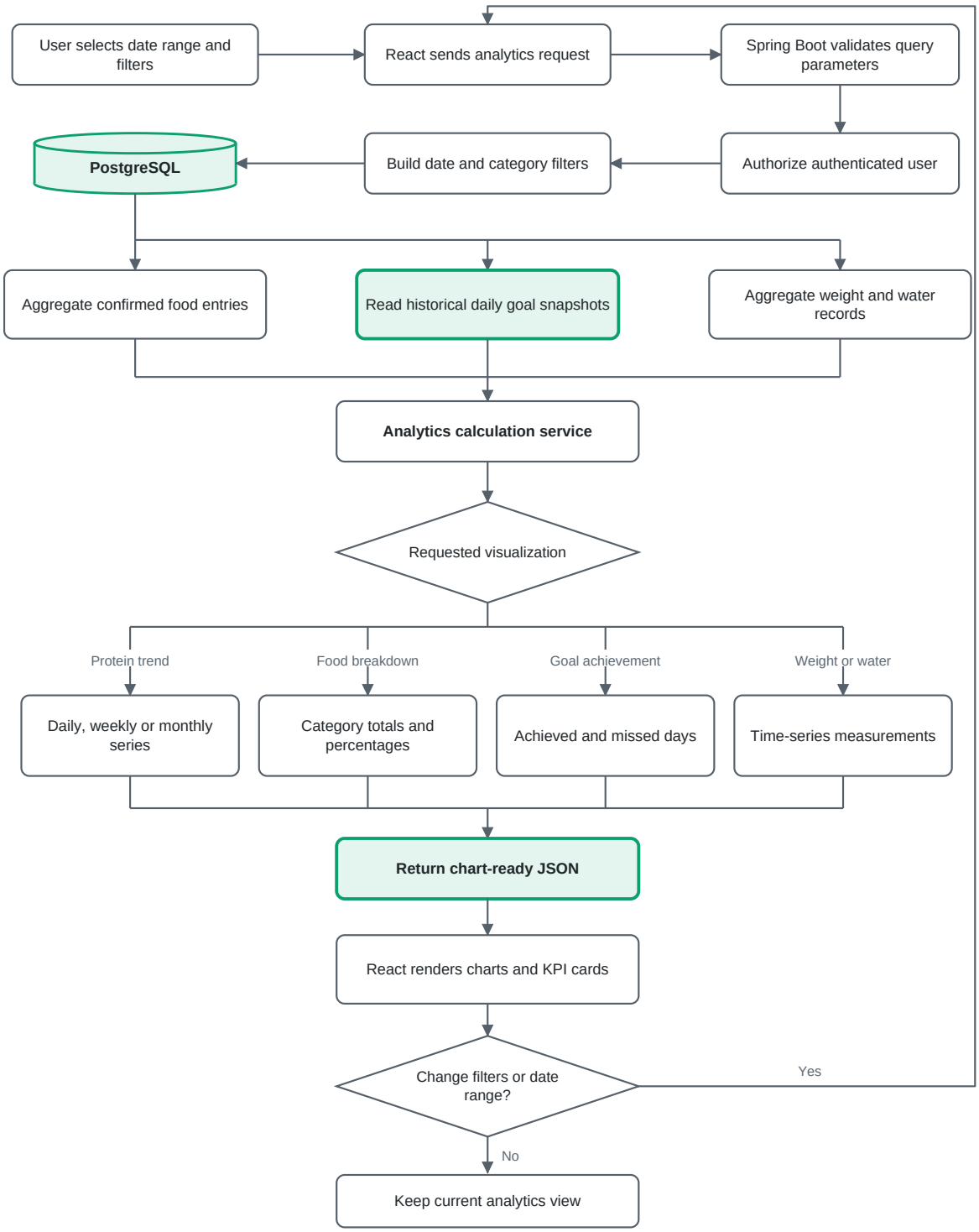

Figure 4: Daily log totals, streak evaluation and analytics rollups

2.5 Reminders & notification system flow

The notification engine checks active reminder preferences, verifies recent intake, and dispatches Web Push notifications with delivery auditing.

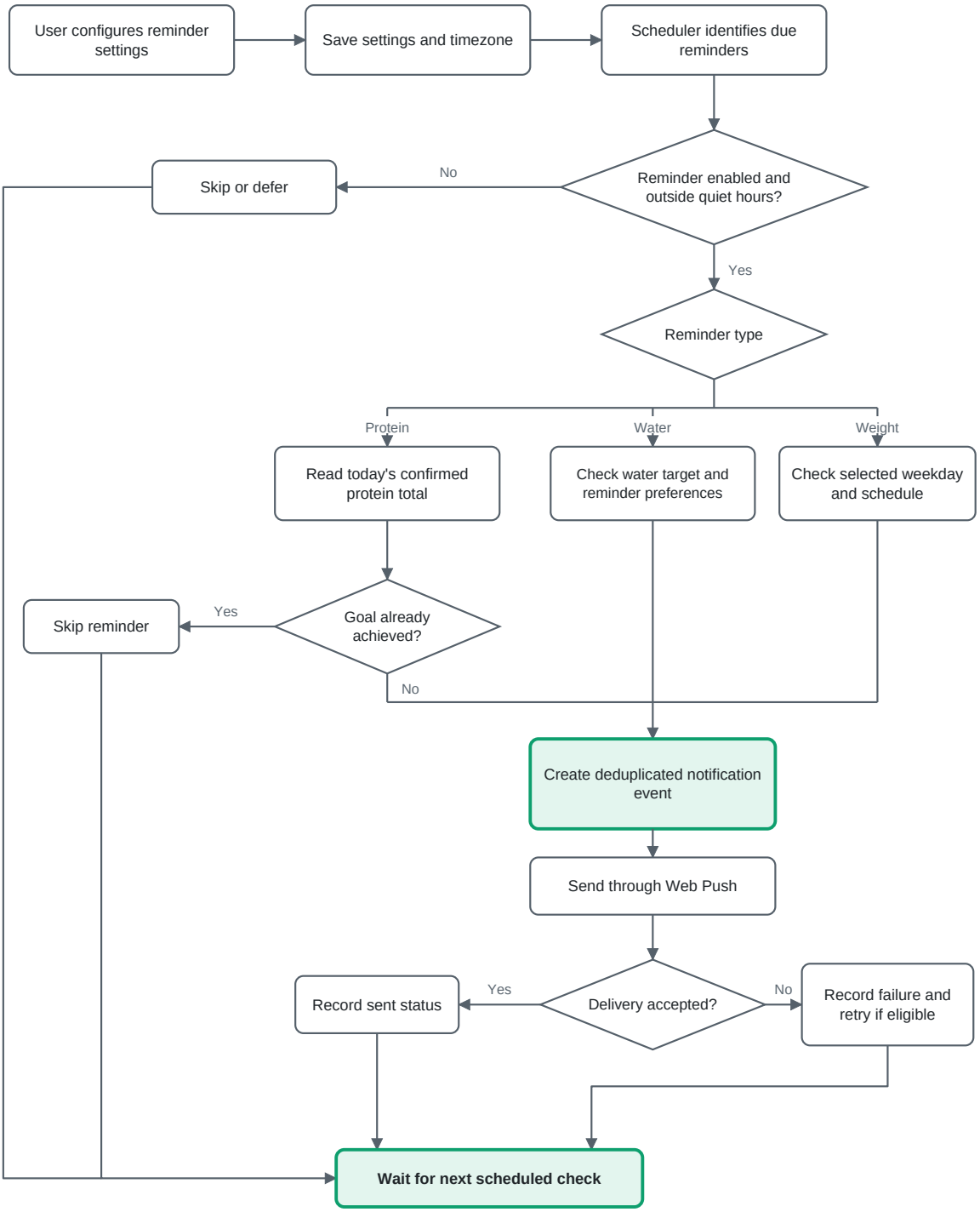

Figure 5: Scheduled reminder evaluation and Web Push dispatch flow

3. Backend File-by-File Catalog

Every file in the backend repository, with its role, key members and architectural connections.

3.1 Project root, build & environment files

File	Role	Description & responsibilities
pom.xml	Build descriptor	Declares all dependencies (Spring Boot, Security, JPA, Flyway, Postgres, Validation, Lombok), compiler release 25, and plugins.
mvnw	Maven Wrapper (POSIX)	Shell script that lets Unix, Linux and macOS developers run Maven identically without a pre-installed Maven binary.
mvnw.cmd	Maven Wrapper (Windows)	Batch script for building and running Maven commands on Windows PowerShell or CMD.
.mvn/wrapper/maven-wrapper.properties	Wrapper config	Configures the wrapper distribution to fetch Apache Maven 3.9.9 automatically.
.env.example	Configuration template	Documents the required environment variables: PORT, DB_URL, DB_USERNAME, DB_PASSWORD, CORS_ALLOWED_ORIGINS, COOKIE_SECURE.
.gitignore	Version control	Excludes Maven target directories, IDE settings (.idea, .vscode), temporary scratch scripts and local .env files.
.gitattributes	Version control	Normalizes line endings across platforms (LF on shell scripts, CRLF/auto on source files).
README.md	Documentation	Repository overview, tech stack, local development prerequisites and build steps.
HELP.md	Reference	Auto-generated Spring Boot reference with links to the official documentation.

3.2 Application entry point & configuration

Class	Package	Description & responsibilities
ProtaApplication.java	com.prota	Main application runner containing main(String[] args). Initializes the Spring context, triggers Flyway and starts Tomcat.
CorsProperties.java	com.prota.config	Strongly typed configuration bean bound to app.cors in application.yml. Exposes allowedOrigins, methods, headers and maxAge.
JacksonConfig.java	com.prota.config	Configures the primary ObjectMapper. Registers JavaTimeModule, disables timestamp serialization and enforces UTC timestamps.

3.3 Security layer (com.prota.security)

Class	Role	Description & responsibilities
SecurityConfig.java	Security filter chain	Configures HttpSecurity: session fixation changeSessionId, CookieCsrfTokenRepository (path /), CORS, 401 entry point and route authorization.
SecurityUtils.java	Identity helper	Static helper that extracts the current user UUID (getCurrentUserId()) and email from SecurityContextHolder. Enforces server-side identity.
UserPrincipal.java	UserDetails adapter	Wraps the User entity and implements Spring Security UserDetails (authorities, enabled status, password hash).
CustomUserDetailsService.java	User loader	Implements UserDetailsService. Queries UserRepository by normalized email to supply a UserPrincipal during authentication.

3.4 REST controllers (com.prota.controller)

Controller	Routes	Description & responsibilities
AuthController.java	/api/v1/auth/*	POST /register, POST /login, POST /logout, GET /me, GET /csrf. Coordinates authentication, session cookies and token exchange.
UserController.java	/api/v1/users/*	GET /me, PATCH /me. Returns profile information and updates the full name. Enforces email immutability.
SettingsController.java	/api/v1/settings	GET /, PATCH /. Reads and updates nutrition targets, timezone, weight unit and theme.

3.5 Service layer (com.prota.service)

Service	Key transactions	Description & responsibilities
AuthService.java	register, login, logout, getCurrentUser	Transactional user creation with default user_settings, BCrypt hashing, session fixation rotation and session invalidation.
UserService.java	getProfile, updateProfile	Retrieves the authenticated profile and modifies the full name. Rejects email changes.
SettingsService.java	getSettings, updateSettings	Reads and updates goal preferences. Validates the timezone with ZoneId.of. Guarantees historical DailyLog records are preserved.
DailyFoodLoggingService.java	getOrCreateDailyLog, addFoodEntry, updateFoodEntry, deleteFoodEntry	Creates daily logs, logs meal entries and recalculates consumed protein and calories dynamically.

3.6 JPA entities & enums (com.prota.entity)

Entity	Database table	Description & key attributes
User.java	users	Core user entity. UUID PK, fullName, email, passwordHash, status, emailVerifiedAt, timestamps.
UserSetting.java	user_settings	One-to-one preferences: userId (PK/FK), proteinGoalG, calorieGoalKcal, waterGoalMl, weightUnit, timezone, theme.
DailyLog.java	daily_logs	Per-date record with frozen goal snapshots (proteinGoalG, calorieGoalKcal). Unique (user_id, log_date).
FoodEntry.java	food_entries	Logged meal: foodName, mealType, quantity, unit, proteinG, caloriesKcal, consumedAt, user and dailyLog FKs.
SavedFood.java	food_library_items	Reusable food library items: name, brand, servingSize, servingUnit, proteinG, caloriesKcal, isVerified.
AiFoodEstimate.java	ai_usage_logs	Tracks Gemini interactions: rawInputText, imageUrl, estimatedProteinG, status, tokenUsage.
WaterEntry.java	daily_logs (ext)	Hydration entries: volume consumed, dailyLogId and timestamps.
WeightEntry.java	user_settings (ext)	Bodyweight tracking: weighDate, weightValue, weightUnit, userId.
ReminderSetting.java	notification_preferences	Notification reminders: reminderType, scheduleType, scheduledTime, isEnabled, channel.
NotificationEvent.java	security_audit_logs	Audit log of dispatched notifications: recipient, title, body, status, sentAt.
PushSubscription.java	notification_preferences	Web Push subscription: endpoint, p256dhKey, authKey, userAgent.
RefreshToken.java	refresh_tokens	Multi-device session tokens: tokenHash, expiresAt, revokedAt (reserved for future token rotation).

3.7 Spring Data JPA repositories (com.prota.repo)

Repository	Managed entity	Key query methods
UserRepository.java	User	findByEmail(String), existsByEmail(String)
UserSettingRepository.java	UserSetting	findByUserId(UUID)
DailyLogRepository.java	DailyLog	findByUserIdAndLogDate(UUID, LocalDate), existsByUserIdAndLogDate(...)
FoodEntryRepository.java	FoodEntry	findAllByDailyLogId(UUID), findByIdAndUserId(UUID, UUID)
SavedFoodRepository.java	SavedFood	findAllByUserIdOrUserIdIsNull(UUID), searchByName(String)
AiFoodEstimateRepository.java	AiFoodEstimate	findAllByUserIdOrderByCreatedAtDesc(UUID)
WaterEntryRepository.java	WaterEntry	findAllByUserIdAndDate(...)
WeightEntryRepository.java	WeightEntry	findAllByUserIdOrderByWeighDateDesc(UUID)
ReminderSettingRepository.java	ReminderSetting	findAllByUserIdAndIsEnabledTrue(UUID)
NotificationEventRepository.java	NotificationEvent	findAllByUserIdOrderByCreatedAtDesc(UUID)
PushSubscriptionRepository.java	PushSubscription	findAllByUserId(UUID)
RefreshTokenRepository.java	RefreshToken	findByTokenHash(String)

3.8 Data transfer objects & exception handling

Class	Category	Description & payload attributes
RegisterRequest.java	Request DTO	fullName, email (auto-trimmed), password. Jakarta validation: @NotBlank, @Email, @Size(8..100).
LoginRequest.java	Request DTO	email (auto-trimmed), password. Validates credentials.
UpdateProfileRequest.java	Request DTO	fullName. Validates non-blank and a maximum of 100 characters.
UpdateSettingsRequest.java	Request DTO	proteinGoalG (>= 1.00), calorieGoalKcal (> 0), waterGoalMl (> 0), weightUnit, timezone, theme.
CreateFoodEntryRequest.java	Request DTO	foodName, mealType, quantity, quantityUnit, proteinG, caloriesKcal, consumedAt.
UpdateFoodEntryRequest.java	Request DTO	Changes to the quantities or nutritional values of an existing food entry.
UserResponse.java	Response DTO	id, fullName, email, status, emailVerifiedAt, createdAt. Excludes password hashes.
SettingsResponse.java	Response DTO	userId, proteinGoalG, calorieGoalKcal, waterGoalMl, weightUnit, timezone, theme, updatedAt.
DailyLogResponse.java	Response DTO	id, logDate, goals, consumedProteinG, proteinRemainingG, completion %, foodEntries list.
CsrfResponse.java	Response DTO	token, headerName, parameterName. Used by the SPA for CSRF bootstrap.
ErrorResponse.java	Response DTO	Standard RFC 9457 Problem Details envelope (code, message, status, timestamp, fieldErrors).
GlobalExceptionHandler.java	Advice	Translates exceptions (validation, conflicts, missing auth) into uniform RFC 9457 JSON.

3.9 Resources, migrations & test suites

File path	Category	Description & highlights
application.yml	Resource	Database settings, context path (/api/v1), Hikari pool, session cookie (PROTASESSIONID, http-only), CORS.
application-dev.yml	Resource	Dev profile with SQL formatting and DEBUG logging.
V1__create_initial_schema.sql	Flyway migration	Creates all 12 tables, composite keys, indexes and constraints.
Phase2AAuthAndProfileTests.java	Integration test	10 tests: registration, duplicate email, password hash check, login/logout, CSRF, user isolation, snapshot preservation.
DatabaseSchemaAndConstraintTests.java	Integration test	6 tests: verifies all 12 tables, UUID primary keys, composite keys and constraints.
SecurityAndActuatorTests.java	Integration test	4 tests: actuator health, sensitive actuators, 401 for unauthenticated requests, CORS headers.

4. API Contract Reference

All routes live under the context path /api/v1. State-changing requests require the CSRF header.

4.1 Authentication endpoints

POST `/api/v1/auth/register`

Registers a new account with BCrypt password hashing, default settings and session cookie issuance.

```
// POST /api/v1/auth/register
{
  "fullName": "Jane Doe",
  "email": "jane@example.com",
  "password": "SecurePassword123!"
}

// 201 Created
{
  "id": "c0fcf568-338f-43c8-a1d4-ad1ad40a3f9f",
  "fullName": "Jane Doe",
  "email": "jane@example.com",
  "status": "ACTIVE",
  "emailVerifiedAt": null,
  "createdAt": "2026-10-20T08:22:34.645Z"
}
```

POST `/api/v1/auth/login`

Authenticates credentials, changes the session ID (fixation protection) and sets PROTASESSIONID.

```
// POST /api/v1/auth/login
{
  "email": "jane@example.com",
  "password": "SecurePassword123!"
}
```

POST `/api/v1/auth/logout`

Invalidates the session and clears SecurityContextHolder (HTTP 204 No Content).

GET `/api/v1/auth/csrf`

Returns the active CSRF token for the single-page app.

```
// 200 OK
{
  "token": "d7b93198-10b2-4d22-9df7-23456789abcd",
  "headerName": "X-XSRF-TOKEN",
  "parameterName": "_csrf"
}
```

4.2 Profile & settings endpoints

PATCH `/api/v1/users/me`

Updates the full name (CSRF required). Email is immutable.

```
// PATCH /api/v1/users/me
{
  "fullName": "Jane Smith"
}
```

PATCH `/api/v1/settings`

Updates goal targets and preferences. Historical DailyLog records are preserved.

```
// PATCH /api/v1/settings
{
  "proteinGoalG": 180.00,
  "calorieGoalKcal": 2400.00,
  "waterGoalMl": 3000,
  "weightUnit": "KG",
  "timezone": "America/New_York",
  "theme": "DARK"
}
```

5. Error Handling & RFC 9457 Specification

Every endpoint returns errors in this envelope.

```
{
  "code": "VALIDATION_ERROR",
  "message": "Invalid request payload",
  "status": 400,
  "timestamp": "2026-10-20T08:21:33.427Z",
  "fieldErrors": [
    {
      "field": "proteinGoalG",
      "message": "Protein goal must be at least 1 gram",
      "rejectedValue": -10.00
    }
  ]
}
```

6. Developer Setup & Test Verification

All 21 automated integration tests run directly against PostgreSQL and pass.

```
# Run all integration tests
.\mvnw.cmd test

# Compile and package the production JAR
.\mvnw.cmd package
```

7. Open Items

Points to confirm before the next phase.

Item	What to check
Entity-to-table names	Section 3.6 maps entities to ai_usage_logs, food_library_items, notification_preferences and security_audit_logs, and lists WaterEntry and WeightEntry against daily_logs and user_settings. The architecture spec uses ai_food_estimates, saved_foods, reminder_settings, push_subscriptions, notification_events, water_entries and weight_entries. Confirm the names in V1__create_initial_schema.sql.
Java version wording	Java 25 is an LTS release; Java 26 is not. Consider stating the exact version the build uses.
Refresh tokens	Figure 2 shows refresh-token rotation, but RefreshToken is reserved for a future release. Today sessions rely on the server-side cookie.
Flows not yet in the catalog	The AI, analytics and reminder flows in section 2 have no controllers or services listed in section 3.

8. Final Word

Phase 2A gives Prota a secure foundation: HTTP-only session cookies, double-submit CSRF, BCrypt hashing, user-scoped queries, a Flyway-managed 12-table schema, and 21 integration tests that pass. Goal changes never touch historical DailyLog snapshots, which is the rule the later analytics depends on.

The next step is to build the AI, analytics and reminder flows in section 2 on the same rules: every query scoped to the signed-in user, every total computed on the server, and every AI estimate confirmed by the user before it counts.